

Guía ampliada de clase

List comprehension, diccionarios y funciones auxiliares en Python

Propósito	Servir como material de estudio y consulta para resolver problemas usando estructuras de datos y funciones de apoyo.
Curso	Fundamentos de Programación Imperativa
Nivel	Primer semestre de Ingeniería de Sistemas
Enfoque	Comprender antes de compactar: primero lógica clara, luego sintaxis más expresiva.

Idea central: list comprehension permite crear listas de forma compacta; los diccionarios permiten asociar claves con valores; las funciones auxiliares permiten dividir problemas complejos en tareas simples.

1. Antes de empezar: ¿qué problema estamos resolviendo?

En las últimas clases se ha pasado de escribir instrucciones sueltas a organizar programas mediante funciones. Ahora se agregan tres herramientas que permiten escribir soluciones más limpias y expresivas:

- List comprehension: crear nuevas listas a partir de otras listas o secuencias.
- Diccionarios: almacenar relaciones clave-valor, muy útiles para conteos, búsquedas y agrupaciones.
- Funciones auxiliares: dividir el problema en tareas pequeñas y reutilizables.

Estos temas no deben entenderse como trucos de sintaxis. Son herramientas para resolver problemas de manera más ordenada.

2. List comprehension

2.1 ¿Qué es?

Una list comprehension es una forma compacta de construir una lista nueva a partir de una secuencia existente. Es equivalente a usar un ciclo for y, opcionalmente, una condición.

```
# Forma general
[nueva_expresion for elemento in secuencia]

# Forma con condición
[nueva_expresion for elemento in secuencia if condicion]
```

2.2 Ejemplo base: construir cuadrados

Supongamos que se tiene una lista de números y se desea construir otra lista con el cuadrado de cada elemento.

Forma tradicional	Con list comprehension
-------------------	------------------------

```

numeros = [1, 2, 3, 4]
cuadrados = []

for n in numeros:
    cuadrados.append(n * n)

print(cuadrados)

```

```

numeros = [1, 2, 3, 4]

cuadrados = [n * n for n in numeros]

print(cuadrados)

```

Ambas versiones generan:

```
[1, 4, 9, 16]
```

2.3 Ejemplo con condición: seleccionar pares

También se puede crear una nueva lista con solo algunos elementos, siempre que cumplan una condición.

```

numeros = [1, 2, 3, 4, 5, 6]

pares = [n for n in numeros if n % 2 == 0]

print(pares) # [2, 4, 6]

```

2.4 Cómo leer una list comprehension

La expresión:

```
[n for n in numeros if n % 2 == 0]
```

se puede leer así:

- Tome cada n dentro de numeros.
- Verifique si $n \% 2 == 0$.
- Si la condición se cumple, agregue n a la nueva lista.

2.5 Transformar datos

La parte inicial no tiene que ser el mismo elemento. Puede ser una transformación.

```

palabras = ["hola", "python", "clase"]

longitudes = [len(palabra) for palabra in palabras]

print(longitudes) # [4, 6, 5]

texto = "programacion"

mayusculas = [letra.upper() for letra in texto]

print(mayusculas)
# ['P', 'R', 'O', 'G', 'R', 'A', 'M', 'A', 'C', 'I', 'O', 'N']

```

2.6 Usar list comprehension con funciones

Una list comprehension puede llamar funciones auxiliares. Esto permite que la comprensión sea compacta, pero mantenga lógica clara.

```

def limpiar_palabra(palabra):
    return palabra.lower()

palabras = ["Hola", "MUNDO", "Python"]

normalizadas = [limpiar_palabra(p) for p in palabras]

print(normalizadas) # ['hola', 'mundo', 'python']

```

2.7 ¿Cuándo usarla y cuándo no?

Usar list comprehension cuando...	Evitarla cuando...
La lógica es corta, clara y fácil de leer.	La condición es muy larga o tiene muchos pasos.
Se construye una lista nueva a partir de otra.	Se necesitan muchas instrucciones dentro del ciclo.
El resultado no requiere modificar muchas variables externas.	Se pierde claridad y el código se vuelve difícil de explicar.

Regla de oro: si no puedes explicar la list comprehension en voz alta, es mejor escribirla con un ciclo for tradicional.

2.8 Errores frecuentes

- Confundir filtrado con transformación: `[n*n for n in numeros]` transforma; `[n for n in numeros if n > 0]` filtra.
- Olvidar la condición: si se desea seleccionar solo algunos elementos, se necesita el `if` final.
- Usarla para todo: no siempre una línea más corta significa mejor código.
- Intentar hacer demasiadas cosas: si la expresión es difícil de leer, conviene usar ciclos normales.

3. Diccionarios

3.1 ¿Qué es un diccionario?

Un diccionario es una estructura de datos que permite almacenar información en forma de pares:

clave -> valor

En Python se escriben usando llaves {}. Cada clave se asocia con un valor.

```
persona = {  
    "nombre": "Ana",  
    "edad": 18,  
    "programa": "Ingeniería de Sistemas"  
}  
  
print(persona["nombre"]) # Ana
```

3.2 Diferencia entre lista y diccionario

Lista	Diccionario
Acceso por posición numérica.	Acceso por clave.
Ejemplo: notas[0]	Ejemplo: persona['nombre']
Útil cuando importa el orden o la posición.	Útil cuando se quiere asociar un dato con un nombre o identificador.

3.3 Operaciones básicas

```
persona = {}  
  
# Agregar datos  
persona["nombre"] = "Carlos"  
persona["edad"] = 19  
  
# Consultar datos  
print(persona["nombre"])  
  
# Modificar  
persona["edad"] = 20  
  
# Verificar si una clave existe  
if "edad" in persona:  
    print("La edad está registrada")
```

3.4 Recorrer un diccionario

Existen varias formas de recorrer un diccionario.

```
notas = {  
    "Ana": 4.5,  
    "Luis": 3.8,  
    "Marta": 4.0  
}  
  
# Recorrer claves  
for nombre in notas:  
    print(nombre, notas[nombre])  
  
# Recorrer clave y valor directamente  
for nombre, nota in notas.items():  
    print(nombre, nota)
```

3.5 Uso fundamental: conteo de frecuencias

Uno de los usos más importantes de los diccionarios es contar cuántas veces aparece algo.

```
def contar_letras(texto):
    conteo = {}

    for letra in texto:
        if letra in conteo:
            conteo[letra] += 1
        else:
            conteo[letra] = 1

    return conteo

print(contar_letras("banana"))
# {'b': 1, 'a': 3, 'n': 2}
```

La lógica es:

- Si la letra ya existe como clave, se incrementa su contador.
- Si la letra no existe, se crea con valor inicial 1.

3.6 Conteo de palabras

Este ejemplo es muy útil porque combina listas, cadenas y diccionarios.

```
def contar_palabras(palabras):
    conteo = {}

    for palabra in palabras:
        palabra = palabra.lower()

        if palabra in conteo:
            conteo[palabra] += 1
        else:
            conteo[palabra] = 1

    return conteo

datos = ["Hola", "mundo", "hola", "Python", "mundo"]
print(contar_palabras(datos))
# {'hola': 2, 'mundo': 2, 'python': 1}
```

3.7 Diccionarios para agrupar

Además de contar, los diccionarios pueden agrupar datos. Por ejemplo, agrupar números en pares e impares.

```
def agrupar_paridad(numeros):
    grupos = {
        "pares": [],
        "impares": []
    }

    for n in numeros:
        if n % 2 == 0:
            grupos["pares"].append(n)
        else:
            grupos["impares"].append(n)

    return grupos

print(agrupar_paridad([1, 2, 3, 4, 5, 6]))
# {'pares': [2, 4, 6], 'impares': [1, 3, 5]}
```

3.8 Errores comunes con diccionarios

- Acceder a una clave inexistente: si se usa `diccionario[clave]` y la clave no existe, el programa genera error.
- Confundir clave con valor: en `{'Ana': 4.5}`, la clave es 'Ana' y el valor es 4.5.
- No inicializar el contador: antes de sumar, la clave debe existir o se debe crear.

- Usar una lista cuando el problema pide búsquedas por nombre o conteos: muchas veces un diccionario es más claro.

3.9 Alternativa útil: get()

El método get() permite obtener un valor si la clave existe o un valor por defecto si no existe.

```
def contar_letras_get(texto):  
    conteo = {}  
  
    for letra in texto:  
        conteo[letra] = conteo.get(letra, 0) + 1  
  
    return conteo  
  
print(contar_letras_get("banana"))
```

Para primer acercamiento, es válido usar if/else. get() es una forma más compacta, pero primero debe entenderse la lógica del conteo.

4. Funciones auxiliares

4.1 ¿Qué son?

Una función auxiliar es una función pequeña que resuelve una parte específica de un problema más grande. No necesariamente es la función principal del programa, pero ayuda a organizar la solución.

Idea clave: una función auxiliar debe hacer una sola tarea y hacerla bien.

4.2 ¿Por qué usarlas?

- Hacen el código más claro.
- Permiten reutilizar lógica.
- Facilitan probar partes del programa por separado.
- Evitan repetir condiciones o recorridos.
- Permiten nombrar ideas importantes del problema.

4.3 Ejemplo: contar vocales

Primero pensemos en el problema sin función auxiliar.

```
def contar_vocales(texto):
    contador = 0

    for letra in texto:
        if letra in "aeiou":
            contador += 1

    return contador
```

Ahora podemos separar la pregunta “¿esta letra es vocal?” en una función auxiliar.

```
def es_vocal(letra):
    return letra in "aeiou"

def contar_vocales(texto):
    contador = 0

    for letra in texto:
        if es_vocal(letra):
            contador += 1

    return contador
```

La segunda versión es más expresiva porque el código se lee casi como una frase: si es vocal, aumentar contador.

4.4 Ejemplo: validar si un número es positivo

```
def es_positivo(numero):
    return numero > 0

def contar_positivos(lista):
    contador = 0

    for numero in lista:
        if es_positivo(numero):
            contador += 1

    return contador

print(contar_positivos([3, -1, 0, 5, 8]))
# 3
```

4.5 Funciones auxiliares con matrices

Cuando se recorren matrices, las funciones auxiliares ayudan a separar la condición del recorrido.

```
def es_par(numero):
    return numero % 2 == 0

def contar_pares_matriz(matriz):
    contador = 0

    for fila in matriz:
        for elemento in fila:
            if es_par(elemento):
                contador += 1

    return contador

m = [
    [1, 2, 3],
    [4, 5, 6]
]

print(contar_pares_matriz(m))
# 3
```

4.6 Ejemplo: una función llama a otra

Las funciones pueden llamarse entre sí. Esto permite construir soluciones por capas.

```
def normalizar_palabra(palabra):
    return palabra.lower()

def contar_palabras(palabras):
    conteo = {}

    for palabra in palabras:
        palabra = normalizar_palabra(palabra)

        if palabra in conteo:
            conteo[palabra] += 1
        else:
            conteo[palabra] = 1

    return conteo

datos = ["Hola", "hola", "Mundo"]
print(contar_palabras(datos))
# {'hola': 2, 'mundo': 1}
```

4.7 Función auxiliar vs función principal

Función auxiliar	Función principal
Resuelve una tarea pequeña.	Coordina varias tareas.
Suele ser reutilizable.	Suele organizar el flujo del programa.
Ejemplo: es_vocal(letra)	Ejemplo: main()

4.8 Buenas prácticas al crear funciones auxiliares

- Usar nombres claros: es_par, contar_vocales, normalizar_palabra.
- Evitar funciones que hagan demasiadas cosas.
- Preferir return cuando se necesita usar el resultado.
- No mezclar entrada de datos, procesamiento y salida si se puede separar.
- Probar cada función con ejemplos pequeños.

Error frecuente: crear funciones que solo imprimen resultados cuando realmente se necesita retornar un valor. Si el resultado debe usarse después, la función debe usar return.

5. Integración de los tres temas

5.1 Problema integrador: contar palabras normalizadas

Se recibe una lista de palabras. Se desea convertir todas a minúscula y contar cuántas veces aparece cada palabra.

```
def normalizar(palabra):
    return palabra.lower()

def contar_frecuencias(palabras):
    conteo = {}

    for palabra in palabras:
        palabra = normalizar(palabra)

        if palabra in conteo:
            conteo[palabra] += 1
        else:
            conteo[palabra] = 1

    return conteo

palabras = ["Hola", "Mundo", "hola", "PYTHON", "python"]
print(contar_frecuencias(palabras))
# {'hola': 2, 'mundo': 1, 'python': 2}
```

Este ejemplo integra:

- Funciones auxiliares: normalizar.
- Diccionarios: almacenamiento de frecuencias.
- Ciclos: recorrido de la lista.
- Cadenas: conversión con lower().

5.2 Agregando list comprehension

También se puede usar list comprehension para normalizar primero todas las palabras.

```
def normalizar(palabra):
    return palabra.lower()

def contar_frecuencias(palabras):
    palabras = [normalizar(p) for p in palabras]

    conteo = {}

    for palabra in palabras:
        if palabra in conteo:
            conteo[palabra] += 1
        else:
            conteo[palabra] = 1

    return conteo
```

Aquí la list comprehension se usa para transformar la lista de palabras. Luego el diccionario se usa para contar.

5.3 Problema integrador con matrices

Se tiene una matriz de números. Se desea obtener una lista con todos los números positivos y luego contar cuántas veces aparece cada uno.

```
def es_positivo(numero):
    return numero > 0

def extraer_positivos(matriz):
    positivos = []
```

```

for fila in matriz:
    for elemento in fila:
        if es_positivo(elemento):
            positivos.append(elemento)

return positivos

def contar_frecuencias(lista):
    conteo = {}

    for elemento in lista:
        if elemento in conteo:
            conteo[elemento] += 1
        else:
            conteo[elemento] = 1

    return conteo

m = [
    [1, -2, 3],
    [1, 0, 3],
    [4, -1, 3]
]

positivos = extraer_positivos(m)
print(positivos) # [1, 3, 1, 3, 4, 3]
print(contar_frecuencias(positivos)) # {1: 2, 3: 3, 4: 1}

```

5.4 Versión con list comprehension anidada

Cuando ya se entiende bien el recorrido doble, se puede compactar la extracción de positivos.

```

m = [
    [1, -2, 3],
    [1, 0, 3],
    [4, -1, 3]
]

positivos = [
    elemento
    for fila in m
    for elemento in fila
    if elemento > 0
]

print(positivos)
# [1, 3, 1, 3, 4, 3]

```

Esta versión es compacta, pero solo debe usarse si el estudiante comprende que equivale a dos ciclos anidados.

6. Comparación: solución extensa vs solución compacta

No siempre la solución más corta es la mejor. En programación, claridad y corrección son más importantes que escribir menos líneas.

Forma extensa	Forma compacta
Más fácil de depurar paso a paso.	Más breve y expresiva si se entiende bien.
Recomendada cuando se está aprendiendo.	Útil cuando la lógica es simple.
Permite ver contadores, acumuladores y condiciones claramente.	Puede volverse confusa si se abusa de ella.

Criterio docente: primero deben saber resolver el problema con ciclos normales. Luego pueden compactar con list comprehension si la lógica sigue siendo clara.

7. Ejercicios propuestos

7.1 List comprehension

- Dada una lista de números, crear una nueva lista con los números mayores que 10.
- Dada una lista de palabras, crear una lista con la longitud de cada palabra.
- Dada una cadena, crear una lista con sus vocales.
- Dada una lista de notas, crear una lista con las notas aprobadas, es decir, mayores o iguales a 3.0.

7.2 Diccionarios

- Contar cuántas veces aparece cada letra en una cadena.
- Contar cuántas veces aparece cada número en una lista.
- Crear un diccionario donde la clave sea el nombre de un estudiante y el valor sea su nota.
- Agrupar una lista de números en pares e impares usando un diccionario.

7.3 Funciones auxiliares

- Crear una función `es_par(numero)` y usarla para contar pares en una lista.
- Crear una función `es_vocal(letra)` y usarla para contar vocales en una cadena.
- Crear una función `es_positivo(numero)` y usarla para contar positivos en una matriz.
- Crear una función `normalizar(palabra)` y usarla para contar palabras sin diferenciar mayúsculas de minúsculas.

7.4 Ejercicio integrador

Diseñe un programa que reciba una lista de frases y determine cuántas veces aparece cada palabra, sin diferenciar entre mayúsculas y minúsculas. El programa debe:

- Usar al menos una función auxiliar.
- Usar un diccionario para el conteo.
- Usar list comprehension en algún punto de la solución.
- Retornar el diccionario final, no solo imprimirlo.

8. Checklist de estudio

- ¿Puedo explicar una list comprehension en palabras?
- ¿Sé cuándo usar un diccionario en lugar de una lista?
- ¿Entiendo la diferencia entre clave y valor?
- ¿Puedo contar frecuencias usando un diccionario?
- ¿Sé crear funciones auxiliares pequeñas y claras?
- ¿Uso return cuando necesito reutilizar un resultado?
- ¿Puedo resolver primero con ciclos normales antes de compactar?
- ¿Evito escribir código corto pero difícil de entender?

9. Resumen final

Tema	Idea principal	Uso típico
------	----------------	------------

List comprehension	Crear listas de forma compacta.	Filtrar o transformar datos.
Diccionarios	Asociar claves con valores.	Conteo, búsqueda, agrupación.
Funciones auxiliares	Dividir problemas en partes pequeñas.	Reutilizar condiciones o procesos.

Frase final: un buen programador no solo busca que el código funcione; busca que el código se pueda leer, explicar, probar y mejorar.